MediSearch – Intelligent Medical Information Retrieval System

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

V. G. Shanmukha Sai : 2520030341
M. Vignesh : 2520030065

Team Number: _____
Section: 2

Under the Guidance of

Dr. S. Vinay Kumar
Associate Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University)

2026-27

Declaration

We declare that this project report, “MediSearch – Intelligent Medical Information Retrieval System”, is our own academic work for the Data Structures and Algorithms -3 course. The project uses only synthetic patient records.

This project is made only for learning and demonstration. It must not be used for real medical diagnosis or treatment.

Student	Roll Number	Signature
V. G. Shanmukha Sai	2520030341	_____
M. Vignesh	2520030065	_____

Certificate

This is to certify that the mini project titled “MediSearch – Intelligent Medical Information Retrieval System” was completed by the above students as part of the B-Tech Computer Science and Engineering course during the academic year 2026-27.

Signature of the Guide
(Dr. S. Vinay Kumar)

Signature of the HOD

Signature of the External Examiner

Acknowledgement

We sincerely thank Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his guidance and support during this project.

We also thank Dr. P. Venkateshwara Rao, HOD, Department of Computer Science and Engineering, and all faculty members for their support and encouragement.

We thank our team members for their cooperation in coding, testing and completing the project.

Abstract

Medi-Search is a simple medical information search system made for the Data Structures and Algorithms course. It stores synthetic patient records and helps users search and view them easily.

The web application provides login, a dashboard, patient search, filters and patient details. It uses KMP, Rabin–Karp and Suffix Array for searching. The Java application contains several more DSA algorithms for demonstration.

All patient data in this project is synthetic. The system is only for academic learning and must not be used for real medical decisions.

Table of Contents

Chapter	Section	Subsections
1	Introduction	1.1 Overview; 1.2 Problem Statement; 1.3 Scope; 1.4 Objectives
2	System Requirements	2.1 Functional Requirements; 2.2 Non-Functional Requirements; 2.3 Software Environment
3	System Design	3.1 High-Level Architecture; 3.2 Data Model; 3.3 Module Description; 3.4 Data Flow
4	Implementation	4.1 Project Structure; 4.2 Web Dashboard; 4.3 Java DSA Engine; 4.4 Algorithm-wise Implementation; 4.5 Authentication
5	Results	5.1 Dashboard; 5.2 Patient Directory; 5.3 Search Results; 5.4 Algorithm Demonstrations; 5.5 Test Cases
6	Challenges and Limitations	6.1 Challenges; 6.2 Solutions; 6.3 Limitations
7	Conclusion and Future Work	7.1 Conclusion; 7.2 Future Work; 7.3 Skills Learned
	References	

List of Figures

Figure	Title
Fig. 1.1	MediSearch project overview
Fig. 3.1	High-level architecture of MediSearch
Fig. 3.2	Patient search and filtering flow
Fig. 4.1	Java DSA console menu
Fig. 5.1	Dashboard statistics and analytics
Fig. 5.2	Patient directory and algorithm selector

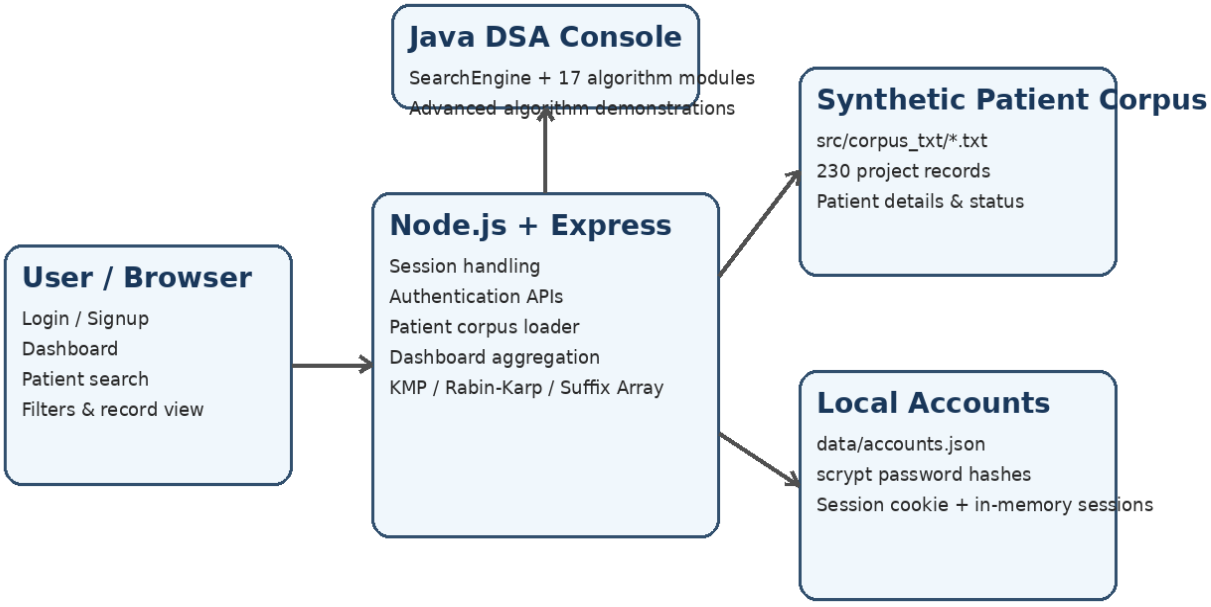
1. INTRODUCTION

1.1 Overview

MediSearch is a local web application for searching synthetic patient records. The main purpose is to show how Data Structures and Algorithms can be used in a practical search system.

The project has 230 synthetic patient text records. A record contains information such as Patient ID, name, age, disease, symptoms, severity, department, status and other details.

Fig. 1.1. MediSearch project overview



1.2 Problem Statement

Finding information manually in many patient records can take time. Different search algorithms can make searching faster and also help us understand how DSA works in a real application.

MediSearch solves this problem by loading the patient records, allowing the user to search them, applying filters and showing the matching records clearly.

1.3 Scope of the Project

- Local patient-record retrieval using synthetic data.
- Account signup, login, logout and session-controlled access to the workspace.
- Dashboard statistics for total patients, urgent records, departments, severity and common diseases.
- Patient-directory search using KMP, Rabin–Karp and suffix-array search.
- Filtering by disease and status.
- Viewing the complete synthetic patient record in a detail dialog.
- Java console demonstrations of 17 algorithmic techniques.
- Educational visualization/exploration of algorithm behavior through project modules.
- No real patient data and no clinical diagnosis or treatment decisions.

1.4 Objectives

1. Build a simple patient-record search system.
2. Use different string-search algorithms on the patient records.
3. Create an easy dashboard for searching and viewing records.
4. Demonstrate graph, optimization, matching, string and other DSA algorithms using Java.
5. Add basic login and password protection to the local application.
6. Make the project easy to explain during the DSA-3 project review and viva.

2. SYSTEM REQUIREMENTS

2.1 Functional Requirements

ID	Requirement	Description
FR-01	User registration	Create a local account using name, email and password.
FR-02	User login	Authenticate an existing account and create a session.
FR-03	Dashboard	Display patient count, urgent count, department count, severity distribution and top diseases.
FR-04	Patient directory	Display patient records in a searchable table.
FR-05	Algorithm selection	Select KMP, Rabin–Karp or suffix-array search in the web dashboard.
FR-06	Filtering	Filter records by disease and patient status.
FR-07	Record details	Open a detailed view of a selected synthetic patient record.
FR-08	Java DSA console	Run additional algorithms using the Java command-line menu.
FR-09	Logout	End the active local session.

2.2 Non-Functional Requirements

- The dashboard is divided into simple sections such as overview, patient directory and search.
- The patient records are loaded when the server starts so that searches can use the loaded data.

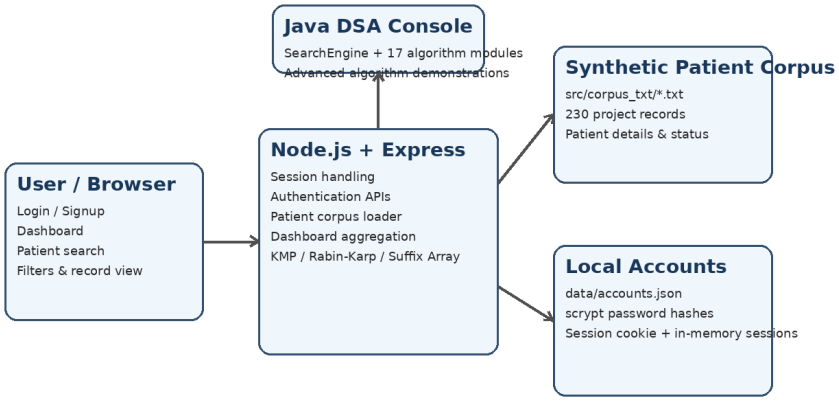
- Passwords are stored as secure hashes using Node.js scrypt. Login sessions use an HTTP-only cookie.
- The server also uses basic security headers.
- Java algorithms are kept in separate classes, while the web application uses separate HTML, CSS and JavaScript files.
- The application clearly tells the user that the data is synthetic and is not for clinical use.

2.3 Software Environment

Component	Used in the project
Java	Patient model, corpus loader, SearchEngine, Main and algorithm classes.
Node.js	Local web server runtime.
Express.js	HTTP server, API routes and static file serving.
HTML	Dashboard and authentication interface.
CSS	Responsive visual design and component styling.
JavaScript	Dashboard interaction, API calls, filtering and rendering.
Text corpus	Synthetic patient records under src/corpus_txt.
JSON	Local account persistence in MediSearch/data/accounts.json.
Browser	Local client interface at 127.0.0.1:3000.

3. SYSTEM DESIGN

3.1 High-Level Architecture Diagram



The browser communicates with the local Express server. The server reads the synthetic patient records and local account data. The Java program is used separately for DSA demonstrations.

3.2 Data Model

The main Patient object stores the important information about one patient record.

Field	Purpose
Patient_ID	Unique identifier for the patient record.
Name	Synthetic patient name.
Age	Patient age.
Disease	Recorded condition in the synthetic dataset.
Symptoms	Recorded symptom text.
Severity	Severity category used by dashboard analytics.
Searchable Text	Full record text used by the Java search engine.
Department	Department associated with the record in the web dashboard.
Status	Current synthetic record status.
Admission Date	Synthetic admission date.
Details	Complete key-value record used for detailed display.

3.3 Module Description

Module	Main Responsibility
Authentication Module	Signup, login, logout, password hashing and sessions.
Corpus Module	Read and parse patient text files.
Dashboard Module	Aggregate patient, severity, department and disease statistics.
Search Module	Run KMP, Rabin–Karp or suffix-array search on patient text.
Filter Module	Apply disease and status filters.
Patient Detail Module	Display all available fields of a selected record.
Java DSA Module	Run the full set of algorithm demonstrations from the console.

3.4 Data Flow

7. The server starts and loads the local accounts and patient records.
8. The user first sees the login or signup page.
9. After login, the dashboard requests patient information and statistics.
10. The server calculates the dashboard statistics from the loaded records.
11. When the user searches, the browser sends the search word and selected algorithm to the server.
12. The selected search algorithm checks the patient records and returns the matching records.
13. The user can open any matching record to see its complete details.
14. The Java program loads the same type of patient corpus and provides more DSA algorithms through its menu.

4. IMPLEMENTATION

4.1 Project Structure

Path / File	Purpose
src/Patient.java	Patient data model.
src/CorpusLoader.java	Loads and parses text patient records.
src/SearchEngine.java	Connects patient records to Java algorithms.
src/Main.java	Java console menu and algorithm execution.
src/algorithms/*.java	Individual DSA implementations.
src/corpus_txt/*.txt	Synthetic patient corpus; current project contains 230 records.
MediSearch/server.js	Express server, authentication, dashboard APIs and web search.
MediSearch/public/index.html	Web dashboard structure.
MediSearch/public/styles.css	Dashboard styling.
MediSearch/public/app.js	Client-side interaction and rendering.
MediSearch/data/accounts.json	Local account persistence.

4.2 Web Dashboard Implementation

The web application has a login page and a main dashboard. The dashboard contains an overview, patient directory and search section. JavaScript is used to communicate with the server and update the page.

- Overview: shows total patients, urgent patients, departments, severity and common diseases.
- Patient Directory: shows patient records in a searchable table.
- Search: the user can select KMP, Rabin–Karp or Suffix Array.
- Patient Details: clicking a record shows its available details.
- Responsive design: the page is designed to work on different screen sizes.

4.3 Java DSA Engine

The Java application loads the patient corpus through SearchEngine. Main.java provides a menu for the different algorithm demonstrations.

4.4 Algorithm-wise Implementation

No.	Algorithm	Project Use / Demonstration	Interface
1	KMP	Exact substring search in patient text.	Web + Java
2	Z Algorithm	Pattern matching using the Z array.	Java
3	Trie	Disease-name lookup using prefix-tree structure.	Java
4	Edit Distance	Find disease names close to a query.	Java
5	Aho–Corasick	Multi-pattern string matching.	Java
6	Smith–Waterman	Local sequence/string similarity scoring.	Java
7	Optimal BST	Disease lookup using frequency-based optimal BST construction.	Java
8	Rabin–Karp	Rolling-hash based substring search.	Web + Java
9	Suffix Array	Lexicographically indexed suffix search.	Web + Java
10	Needleman–Wunsch	Global sequence alignment with match, mismatch and gap scores.	Java
11	Traveling Salesperson	Exact round-trip route optimization for an input distance matrix.	Java
12	Bipartite Matching	Maximum assignment between care teams and patients/appointments.	Java
13	Edmonds–Karp	Maximum-flow and minimum-cut demonstration.	Java
14	Vertex Cover Approximation	2-approximation for an undirected graph.	Java

15	0/1 Knapsack	Select items to maximize value under capacity.	Java
16	Miller–Rabin	Probable-prime test using witness rounds.	Java
17	Reservoir Sampling	One-pass sampling of patient records.	Java

4.5 Authentication and Security Implementation

- Signup checks the basic user details before creating an account.
- Passwords are not saved as normal text. The server uses scrypt to create a password hash.
- After login, a session is created and stored on the server.
- The login session is kept for eight hours.
- Dashboard and patient APIs can be used only after login.
- The server uses basic security headers to improve browser security.
- Only synthetic data should be used in this project.

5. RESULTS

5.1 Dashboard Results

After login, the dashboard shows information about the patient records, such as total patients, urgent records, departments, severity and common diseases.

Fig. 5.1. Dashboard statistics and analytics

The dashboard values are calculated from the patient files currently stored in the project.

5.2 Patient Directory

The patient directory shows patient name, condition, department, severity and status. The user can search, apply filters and open a record to see more details.

5.3 Search Results

Search Method	Web Behavior
KMP	Searches the normalized patient-record text using a prefix-function based pattern search.
Rabin–Karp	Searches using rolling hash and verifies matching windows.
Suffix Array	Builds sorted suffix positions for each record and uses binary search to locate the query.

5.4 Java Algorithm Demonstrations

The Java console is used as a DSA laboratory. The user can enter input and run algorithms for sequence comparison, routes, matching, flow, graphs, knapsack, prime testing and sampling.

5.5 Functional Test Cases

Test ID	Test	Expected/Implemented Behavior

TC-01	Signup with valid details	Account created and workspace opened.
TC-02	Signup with duplicate email	Request rejected with duplicate-account message.
TC-03	Login with valid credentials	Session created and dashboard opened.
TC-04	Login with invalid credentials	Authentication rejected.
TC-05	Open dashboard	Statistics and patient list load.
TC-06	KMP search	Matching patient records are returned.
TC-07	Rabin–Karp search	Matching patient records are returned.
TC-08	Suffix-array search	Matching patient records are returned.
TC-09	Disease/status filter	Only matching records remain.
TC-10	View patient record	Complete synthetic record opens in dialog.
TC-11	Logout	Session cookie is cleared and protected pages require login.
TC-12	Java menu option	Selected DSA algorithm accepts its defined input and prints a result.

6. CHALLENGES AND LIMITATIONS

6.1 Issues Faced During Development

- It was necessary to read all patient files in the same format.
- Different search algorithms had to work with the same patient data.
- Search results and filters had to work together correctly.
- Login had to be added without making the project too complex.
- Many DSA algorithms had to be organized clearly for the project.
- The project had to make it clear that the patient data is only synthetic.

6.2 Solutions Applied

- CorpusLoader is used to read the text files and create Patient objects.
- SearchEngine connects the patient records with the Java algorithms.
- The web server uses one search system and selects KMP, Rabin–Karp or Suffix Array based on the user's choice.
- The dashboard calculates statistics from the loaded patient records.
- Login, signup and logout are handled by separate server functions.
- The interface clearly states that the project is only a demonstration.

6.3 Limitations of the System

- The patient records are synthetic and are not real hospital records.
- Only KMP, Rabin–Karp and Suffix Array are directly available in the web application. The other algorithms are demonstrated in Java.
- Login sessions are stored in server memory, so they end when the server is restarted.
- User accounts are stored in a local JSON file instead of a production database.
- The project is designed for local academic use, not for a real hospital.
- The system does not give diagnosis, treatment or medical advice.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

MediSearch shows how DSA can be used to solve a practical search problem. The system loads synthetic patient records, provides a dashboard, supports search and filters, and allows the user to view records.

The web application demonstrates the main search algorithms, while the Java application provides more algorithms for DSA practice and viva demonstration.

7.2 Future Work

- Use a database for larger datasets.
- Add more DSA algorithms to the web application.
- Show the algorithms step by step using animations.
- Add different user roles if required.
- Add better session and activity logging for future versions.
- Allow safe academic datasets to be imported.
- Add graphs to compare the speed of different search algorithms.
- Add automatic tests for the algorithms and APIs.

7.3 Skills Learned During Development

- Learned how string-search algorithms work.
- Learned Java classes and modular programming.
- Learned graph and optimization algorithms.
- Learned file handling and data parsing.
- Learned Node.js and Express.
- Learned HTML, CSS and JavaScript.
- Learned basic authentication, password hashing and sessions.
- Learned testing, debugging and project documentation.

APPENDIX A – DSA COVERAGE

This table shows where the DSA algorithms are used in MediSearch. It can also be used to explain the project during the viva.

Topic Area	Algorithms / Structures	MediSearch Demonstration
String Matching	KMP, Z Algorithm, Rabin–Karp, Aho–Corasick	Search patient-record text and compare pattern-matching approaches.
Trees / Indexing	Trie, Optimal BST	Disease lookup and frequency-aware disease indexing.
Similarity / Alignment	Edit Distance, Smith–Waterman, Needleman–Wunsch	Approximate disease matching and sequence alignment.
String Indexing	Suffix Array	Lexicographic suffix indexing for patient-record search.
Graph Optimization	TSP, Vertex Cover Approximation	Route and graph optimization demonstrations.
Graph Flow / Matching	Bipartite Matching, Edmonds–Karp	Care-team assignment and network-flow demonstrations.
Dynamic Programming	0/1 Knapsack	Capacity-constrained item selection.
Number Theory	Miller–Rabin	Probable-prime testing.
Randomized Algorithms	Reservoir Sampling	One-pass sampling of patient records.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, MIT Press.

[2] R. Sedgewick and K. Wayne, Algorithms, Addison-Wesley.

[3] Oracle Java Documentation, Java Platform documentation and API reference.

[4] Node.js Documentation, Node.js runtime and standard-library documentation.

[5] Express.js Documentation, Express web framework documentation.

[6] MDN Web Docs, HTML, CSS, JavaScript and Web API documentation.

[7] MediSearch project source code and synthetic patient corpus supplied with this project.

[8] Project Documentation Format 2026-27, Department of Computer Science and Engineering, used as the report structure reference.